

Risk-Adaptive Late-Binding Decryption for Secure Field-Level Data Exposure in Backend Systems

Faiyaz Mahmud, Tirtho Mondal, and Muhammad Aminul Haque Akhand

Department of Computer Science and Engineering
Khulna University of Engineering & Technology (KUET)
Khulna 9203, Bangladesh

Abstract—Application-level encryption protects sensitive database fields at rest, but plaintext can still appear too early when an object-relational mapper decrypts values during entity hydration. This paper presents a thesis-derived proof-of-concept framework that changes the decryption boundary: encrypted fields remain ciphertext after ORM retrieval, and a runtime policy determines whether a request receives full plaintext, partial masked exposure, or denial. Six business features—role, resource sensitivity, office-hour status, record-ownership match, 60-second request count, and 24-hour failed-login count—are encoded as an eight-dimensional vector and scored by a 75-tree Random Forest of maximum depth 12. The model was trained on a 10,000-event synthetic access corpus generated with seed 42, 4% random label redraw, and a stratified 70/15/15 train/validation/test split. On the 1,500-event synthetic test partition, the Random Forest achieved 0.9780 accuracy, 0.9777 weighted F1, and 0.9407 macro-F1. It classified 129 of 139 HIGH-risk simulated contexts as HIGH, corresponding to a 92.8% outright interception rate under the defined synthetic threat model. The model was transpiled to plain JavaScript for in-process NestJS/Node.js execution, with prototype warm inference of approximately 49–50 μ s. End-to-end tests verified the intended field-exposure behavior: LOW performs seven decryptions, MEDIUM performs two allowlisted decryptions with masking, and HIGH returns HTTP 403 before any application-level field decryption. The work is presented as a synthetic proof-of-concept and backend integration study, not as real-world attack-detection validation.

Index Terms—late binding, adaptive decryption, field-level encryption, exposure control, Random Forest, backend security

I. INTRODUCTION

Application-level encryption can protect sensitive fields in database storage while still allowing premature plaintext materialization inside the application process. In a common ORM value-transformer pattern, fields are decrypted automatically during entity hydration. Once plaintext exists in memory, later masking or access-response decisions no longer prevent the original data-in-use exposure. The core problem is therefore not the cipher itself, but the ordering between ORM retrieval, contextual risk assessment, and field decryption.

This paper reformulates decryption as a runtime exposure decision. The prototype keeps authentication and static RBAC authorization as mandatory controls, but it adds a second decision layer at the cryptographic boundary. After an authenticated request reaches a protected resource, server-derived context is encoded and classified as LOW, MEDIUM, or HIGH risk. The selected class then controls whether protected fields are fully

decrypted, partially exposed with masking, or denied before decryption.

The work is strictly derived from the thesis experiment. It uses the same 10,000 synthetic access events, seed 42, 4% label redraw, stratified 70/15/15 split, Logistic Regression baseline, 75-tree Random Forest, AES-256-GCM field encryption, NestJS/PostgreSQL/Node.js backend, and LOW/MEDIUM/HIGH policy. The goal of this conference version is to present that study compactly and cautiously, not to introduce unperformed experiments such as grouped splitting, repeated seeds, distribution-shift testing, or full-corpus Python–JavaScript parity.

The main contributions are threefold: (i) a ciphertext-preserving late-binding ORM design in which protected fields are not automatically decrypted on load; (ii) an in-process Random-Forest exposure-control layer that maps authenticated request context to graduated field exposure; and (iii) end-to-end backend verification showing full, partial, and zero-decryption response modes. The classifier is treated as a synthetic policy approximation mechanism rather than a general attack detector.

II. RELATED WORK AND RESEARCH GAP

AES-GCM provides authenticated encryption suitable for field-level protection when nonce and key-management requirements are respected [1]. Database or storage-level encryption protects files, backups, or raw database media [10], [11]. These mechanisms reduce exposure before decryption, but they do not decide how much plaintext an already executing application path should materialize.

RBAC and ABAC provide deterministic access-control structures based on roles, attributes, and policy rules [2], [3]. Zero Trust further emphasizes continuous verification around resources [4]. Risk-adaptive and ML-assisted access-control systems incorporate context and learned scores into authorization decisions [6]–[9]. However, many such systems focus on admission control or post-hoc security analytics rather than controlling the exact point where ciphertext becomes application plaintext.

The gap addressed here is the integration of four elements in a backend system: ciphertext-preserving ORM retrieval, per-request behavioral risk scoring, graduated field-level exposure, and dependency-free in-process model inference. Unlike a static ORM+RBAC baseline, the proposed framework does not decrypt all permitted fields automatically after route-level

TABLE I
RUNTIME FEATURES AND SERVER-SIDE SOURCES

Feature	Encoding	Source
Role	3 one-hot	Verified identity claim
Sensitivity	0/1/2	Domain-declared data tier
Office hours	0/1	Server clock
Owner match	0/1	JWT subject vs. row owner
60-s request count	Integer	Access-request log
24-h failed-login count	Integer	Authentication log

authorization. Instead, it evaluates runtime context at the decryption boundary.

III. PROPOSED FRAMEWORK

A. Thesis-Consistent Runtime Flow

The thesis framework follows this ordering: authenticated request, static authorization/RBAC, ciphertext-preserving ORM retrieval, runtime feature extraction, Random Forest risk classification, LOW/MEDIUM/HIGH exposure policy, and selective decryption/masking/denial. Record-ownership match is intentionally retained as a model feature because the synthetic threat policy uses cross-record access as an IDOR-related risk signal rather than removing all such contexts before ML scoring.

B. Feature Representation

Each access context is represented as

$$\mathbf{x} = [r_c, r_m, r_a, s, h, o, v, f]^T \in \mathbb{R}^8, \quad (1)$$

where r_c, r_m, r_a are one-hot customer, moderator, and administrator role indicators; $s \in \{0, 1, 2\}$ encodes LOW/MEDIUM/HIGH resource sensitivity; h is the office-hours indicator; o is record-owner match; v is the number of protected reads in the previous 60 seconds; and f is the failed-login count in the previous 24 hours. The same static index mapping is used in Python during training and TypeScript/JavaScript during backend execution.

C. Model and Exposure Policy

The selected model is a Random Forest with $T = 75$ trees, maximum depth 12, balanced class weighting, and random seed 42. For class $c \in \{\text{LOW}, \text{MEDIUM}, \text{HIGH}\}$, the forest score is

$$\hat{p}(c | \mathbf{x}) = \frac{1}{T} \sum_{t=1}^T p_t(c | \mathbf{x}), \quad \hat{c} = \arg \max_c \hat{p}(c | \mathbf{x}). \quad (2)$$

The policy mapping is

$$\begin{aligned} \pi(\text{LOW}) &= \text{FULL}, \\ \pi(\text{MEDIUM}) &= \text{PARTIAL}, \\ \pi(\text{HIGH}) &= \text{DENY}. \end{aligned} \quad (3)$$

The architectural invariant is

$$N_{\text{decrypt}}(q) = 0 \quad \text{whenever } \pi(\hat{c}) = \text{DENY}. \quad (4)$$

In LOW mode, all seven protected fields of the demonstration bank-details record are decrypted. In MEDIUM mode, only branch name and holder name are decrypted; the holder name is masked, the account number is rendered using a separately stored last-four derivative, and higher-value identifiers remain hidden. In HIGH mode, the endpoint returns HTTP 403 before any application-level protected-field decryption.

IV. EXPERIMENTAL METHODOLOGY

A. Synthetic Dataset and Split

The dataset contains 10,000 synthetic access events generated with seed 42. Each event samples role, sensitivity, office-hour status, record ownership, request velocity, and failed-login history. Labels are assigned by explicit domain rules that reflect the synthetic threat model. HIGH examples include customer cross-record access to MEDIUM/HIGH-sensitivity data, unusually high customer request rate, moderator access to HIGH-sensitivity data, bulk cross-record access, off-hours privileged access to high-sensitivity records, and recent failed-login activity. MEDIUM examples represent elevated but subthreshold conditions such as low-sensitivity cross-record access, moderate request velocity, or unusual off-hours access.

A 4% random label redraw introduces ambiguity. The corpus is split using stratified 70/15/15 train/validation/test partitioning, resulting in 7,000 training events, 1,500 validation events, and 1,500 test events. The test set contains 1,274 LOW, 87 MEDIUM, and 139 HIGH samples. This split is the thesis protocol; no grouped split or repeated-seed rerun is reported.

B. Baselines, Transpilation, and Runtime Setup

The thesis compares the selected Random Forest against a Logistic Regression model trained with the same feature representation and partitioning protocol. Logistic Regression achieved an approximately 0.75–0.81 weighted-F1 range, while the Random Forest achieved 0.9777 weighted F1 on the quarantined synthetic test set. The comparison supports the use of a nonlinear tree ensemble for conjunction-shaped synthetic policies, but it should not be read as a broad model-family superiority claim.

The trained Random Forest is transpiled using m2cgen into plain JavaScript for in-process Node.js execution. Transpilation fidelity is verified on representative domain scenarios, including legitimate customer access, a ninth request mapped to MEDIUM, a 40-request/min scraping pattern, a cross-record HIGH-sensitivity IDOR-style context, and eight failed logins. The thesis does not report full 1,500-case Python–JavaScript parity; therefore, no complete-parity claim is made. The backend implementation uses NestJS, PostgreSQL, TypeORM, Node.js, and AES-256-GCM field encryption.

V. RESULTS AND ANALYSIS

A. Classification Results

The Random Forest achieved 0.9780 accuracy and 0.9777 weighted F1 on the 1,500-event held-out same-generator synthetic test set. Table II reports the thesis test metrics, while Fig. 3 visualizes class-wise precision, recall, and F1. The LOW

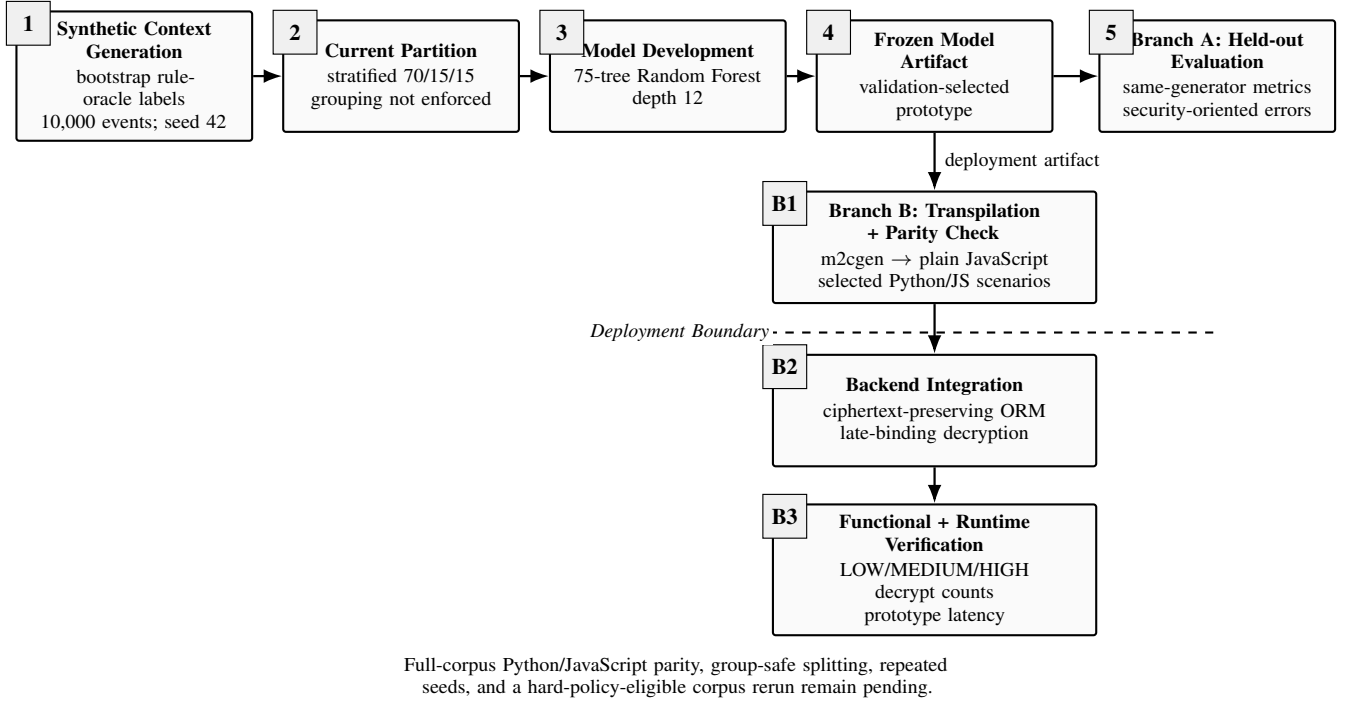


Fig. 1. Experimental and deployment workflow. The frozen model is evaluated on the current held-out synthetic partition and separately transpiled for backend verification. The current split is stratified but not group-safe, so the reported classifier metrics remain same-generator policy-approximation results.

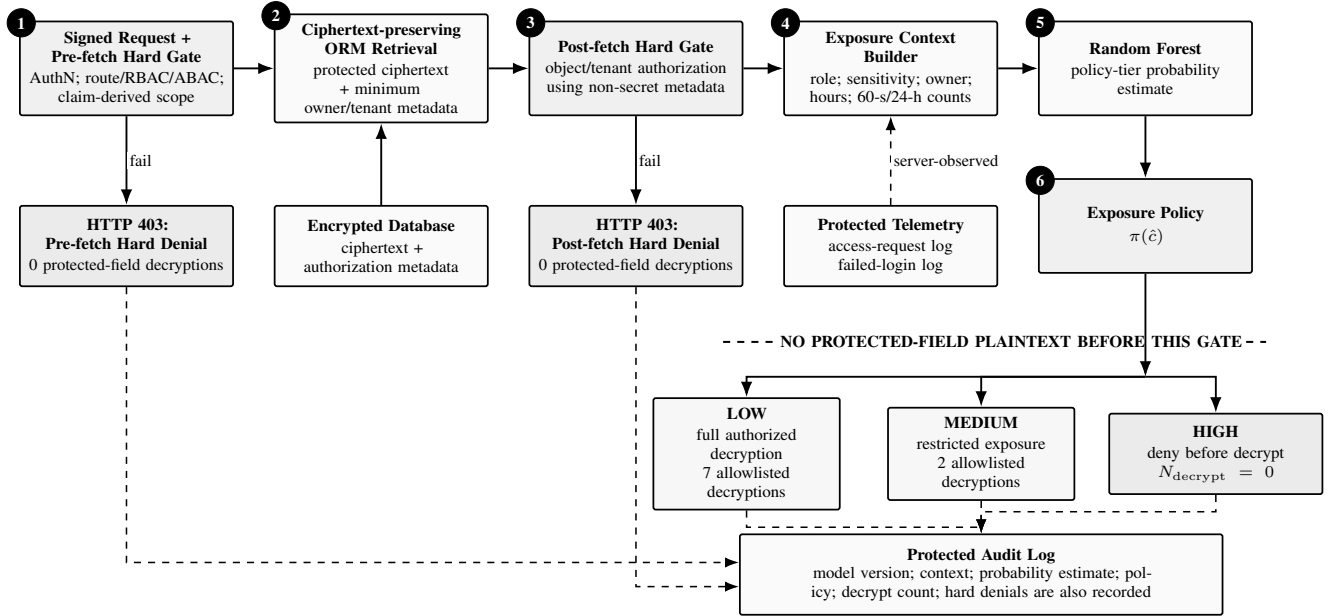


Fig. 2. Runtime enforcement ordering. Ciphertext and minimum non-secret authorization metadata may be retrieved before final object/tenant authorization, but protected fields remain encrypted until the complete hard policy and exposure selection have finished. Pre-fetch denial, post-fetch denial, and the adaptive HIGH branch all terminate before protected-field decryption.

class performs strongest because it dominates the synthetic distribution. MEDIUM recall is the lowest class-wise result at 0.8621, and HIGH recall is 0.9281.

Figure 4 presents the confusion matrix with rows as actual LOW/MEDIUM/HIGH and columns as predicted LOW/MEDIUM/HIGH. Thus, 1,263 of 1,274 LOW samples re-

ceive full access (99.14%), and 129 of 139 HIGH simulated risk contexts are denied outright (92.81%). The nine HIGH→LOW cases and one HIGH→MEDIUM case are security-relevant because they permit more exposure than the synthetic label intends.

TABLE II
HELD-OUT SYNTHETIC TEST PERFORMANCE

Class	Precision	Recall	F1	Support
LOW	0.9836	0.9914	0.9875	1274
MEDIUM	0.9259	0.8621	0.8929	87
HIGH	0.9556	0.9281	0.9416	139
Macro avg.	0.9550	0.9272	0.9407	1500
Weighted avg.	0.9777	0.9780	0.9777	1500

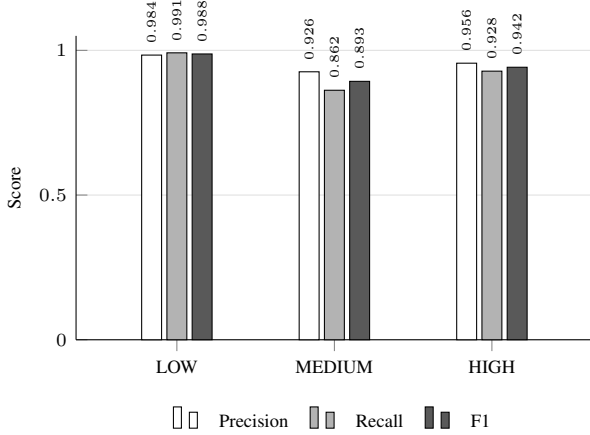


Fig. 3. Per-class performance on the 1,500-event held-out same-generator synthetic test set. MEDIUM recall is the lowest class-wise metric, while HIGH recall is 0.9281.

B. Feature Importance and Runtime Cost

The trained forest assigns the largest MDI importance to 60-second request count (0.4189), sensitivity (0.1744), failed-login count (0.1376), and ownership match (0.1292). The three one-hot role components sum to 0.1040. Fig. 5 is descriptive only; because the same variables help generate the synthetic labels, MDI should not be interpreted as independent causal validation.

Prototype runtime measurements show that model inference is small compared with telemetry retrieval. The transpiled artifact is approximately 3.05 MB. The thesis reports an approximately 50 ms first-call parse/JIT cost, 49–50 μ s warm Random-Forest inference, and approximately 1–2 ms for the two indexed behavioral-count queries. These are prototype measurements rather than p50/p95/p99 latency distributions.

C. End-to-End Field Exposure

The integrated backend verifies the central security behavior. LOW decrypts all seven protected fields. MEDIUM decrypts only two display fields and masks the response; the account-number last-four display is produced from a stored derivative rather than decrypting the protected account number. HIGH returns HTTP 403 with zero application-level protected-field decryptions. Fig. 6 summarizes the field-level exposure behavior.

		LOW	MEDIUM	HIGH
Actual class	LOW	1263 99.14%	5 0.39%	6 0.47%
	MEDIUM	12 13.79%	75 86.21%	0 0.00%
	HIGH	9 6.47%	1 0.72%	129 92.81%

Predicted class

double border = exposure-increasing error toward LOW; percentages are within the actual-class row.

Fig. 4. Row-normalized confusion matrix with counts for the current same-generator synthetic protocol. MEDIUM→LOW and HIGH→LOW errors increase permitted plaintext exposure.

TABLE III
SECURITY-ORIENTED ERROR METRICS

Metric	Value
LOW full-access retention	99.14%
HIGH outright interception	92.81%
HIGH→LOW full-exposure error	6.47%
HIGH→non-HIGH permissive error	7.19%
MEDIUM→LOW overexposure	13.79%
Macro-F1	0.9407

VI. DISCUSSION AND THREATS TO VALIDITY

The main validity threat is that training and testing use a synthetic rule-generated corpus. The Random Forest may therefore learn the labeling policy rather than a generalizable notion of malicious behavior. For this reason, the reported 92.8% HIGH interception rate is stated only under the defined synthetic threat model. It is not evidence of real-world attack-detection effectiveness.

A second limitation is that the thesis uses a single primary seed and a stratified random split rather than grouped or repeated-seed evaluation. Future work should evaluate privacy-preserving operational telemetry, longer-horizon features for low-and-slow behavior, calibrated probability thresholds, distributed counters such as Redis, device and session-level features, broader domains such as healthcare or e-commerce, and staged real-world field trials.

A third limitation concerns deployment validation. Logistic Regression is reported as an approximate range rather than an exact preserved untouched-test value. JavaScript fidelity is verified on representative scenarios rather than every test sample. Runtime evaluation reports prototype point measurements rather than percentile latency under concurrency. These limitations should be fixed in a later extended version, but they are not inserted here as completed experiments.

VII. CONCLUSION

This paper presents a thesis-consistent conference version of a risk-adaptive late-binding decryption framework. The system

Protected field	LOW	MEDIUM	HIGH
Branch name	Plain	Plain	Deny
Account-holder name	Plain	Masked	Deny
Account number	Plain	Last-4*	Deny
Routing number	Plain	Hidden CT	Deny
SWIFT code	Plain	Hidden CT	Deny
IBAN	Plain	Hidden CT	Deny
National identity no.	Plain	Hidden CT	Deny
Decryptions	7	2	0

* Stored low-sensitivity last-four derivative; the protected account-number ciphertext is not decrypted. "Hidden CT" means the protected ciphertext remains hidden from the response.

Fig. 6. Field-level exposure matrix for the same protected record. MEDIUM decrypts only two allowlisted display fields; HIGH denies the request with zero protected-field decryptions.

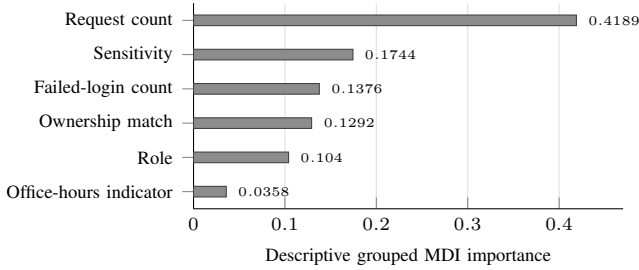


Fig. 5. Descriptive grouped mean-decrease-in-impurity (MDI) importance of the trained Random Forest. The three one-hot role components are summed. MDI is reported only as a fitted-model diagnostic, not as causal or independent feature validation.

TABLE IV
PROTOTYPE RUNTIME CHARACTERIZATION

Property	Reported value
Backend	NestJS; Node.js; PostgreSQL; TypeORM
Model artifact	≈3.05 MB JavaScript; 75 trees
Cold first call	≈50 ms once per process
Warm inference	≈49–50 μs/request
Telemetry counts	≈1–2 ms; indexed queries

preserves ciphertext after ORM retrieval, derives runtime context from server-side evidence, uses a transpiled Random Forest to classify LOW/MEDIUM/HIGH exposure risk, and maps that class to full decryption, partial masked exposure, or denial

before decryption. On the synthetic test partition, the selected Random Forest achieved 0.9780 accuracy and 0.9777 weighted F1, outperforming the thesis Logistic Regression baseline range of approximately 0.75–0.81 weighted F1. End-to-end backend tests further verified the strongest architectural result: LOW, MEDIUM, and HIGH paths perform 7, 2, and 0 protected-field decryptions, respectively. The contribution is a validated proof-of-concept integration pattern for controlling field-level plaintext exposure, with real-world telemetry validation left for future work.

REFERENCES

- [1] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007, doi: 10.6028/NIST.SP.800-38D.
- [2] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Inf. Syst. Secur.*, vol. 4, no. 3, pp. 224–274, Aug. 2001, doi: 10.1145/501978.501980.
- [3] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "Guide to Attribute Based Access Control (ABAC) Definition and Considerations," NIST Special Publication 800-162, updated Aug. 2019, doi: 10.6028/NIST.SP.800-162.
- [4] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, Aug. 2020, doi: 10.6028/NIST.SP.800-207.
- [5] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.
- [6] A. Liu, X. Du, and N. Wang, "Efficient access control permission decision engine based on machine learning," *Security and Communication Networks*, vol. 2021, Art. no. 3970485, 2021, doi: 10.1155/2021/3970485.
- [7] M. Usman, M. S. Sarfraz, U. Habib, M. U. Aftab, and S. Javed, "Automatic hybrid access control in SCADA-enabled IIoT networks using machine learning," *Sensors*, vol. 23, no. 8, Art. no. 3931, 2023, doi: 10.3390/s23083931.
- [8] V. K. Matter, M. G. Martins, and J. L. V. Barbosa, "Context-aware security and machine learning for access control: A systematic mapping and taxonomies," *Computer Science Review*, vol. 60, Art. no. 100880, 2025, doi: 10.1016/j.cosrev.2025.100880.
- [9] R. Prajapati, S. S. Panchal, N. Soni, and S. R. Panchal, "Machine Learning-Based Risk-Adaptive Attribute-Based Access Control Framework for Secure IIoT Networks," *Int. Res. J. Multidisciplinary Technovation*, vol. 8, no. 3, pp. 495–519, 2026, doi: 10.54392/irjmt26329.
- [10] V. Sidorov and W. K. Ng, "Transparent Data Encryption for Data-in-Use and Data-at-Rest in a Cloud-Based Database-as-a-Service Solution," in *Proc. IEEE World Congress on Services*, 2015, doi: 10.1109/SERVICES.2015.40.
- [11] V. Shaik and K. Natarajan, "Flexible and cost-effective cryptographic encryption algorithm for securing unencrypted database files at rest and in transit," *MethodsX*, vol. 9, Art. no. 101924, 2022, doi: 10.1016/j.mex.2022.101924.